Chapter 1. Basic for CSE

한국공학대학교
TECH UNIVERSITY OF KOREA

Lee Youngkon

01 Why Do We Need Object-Oriented Programming?

- Software is Becoming More Complex
  - Modern systems contain:
  - Thousands of functions
  - Millions of lines of code
  - Multiple developers working together
- Examples:
  - ERP Systems
  - Banking Systems
  - E-Commerce Platforms
  - Social Media Applications
- Problem
  - As software grows, maintaining procedural code becomes difficult.

02 Limitations of Procedural Programming

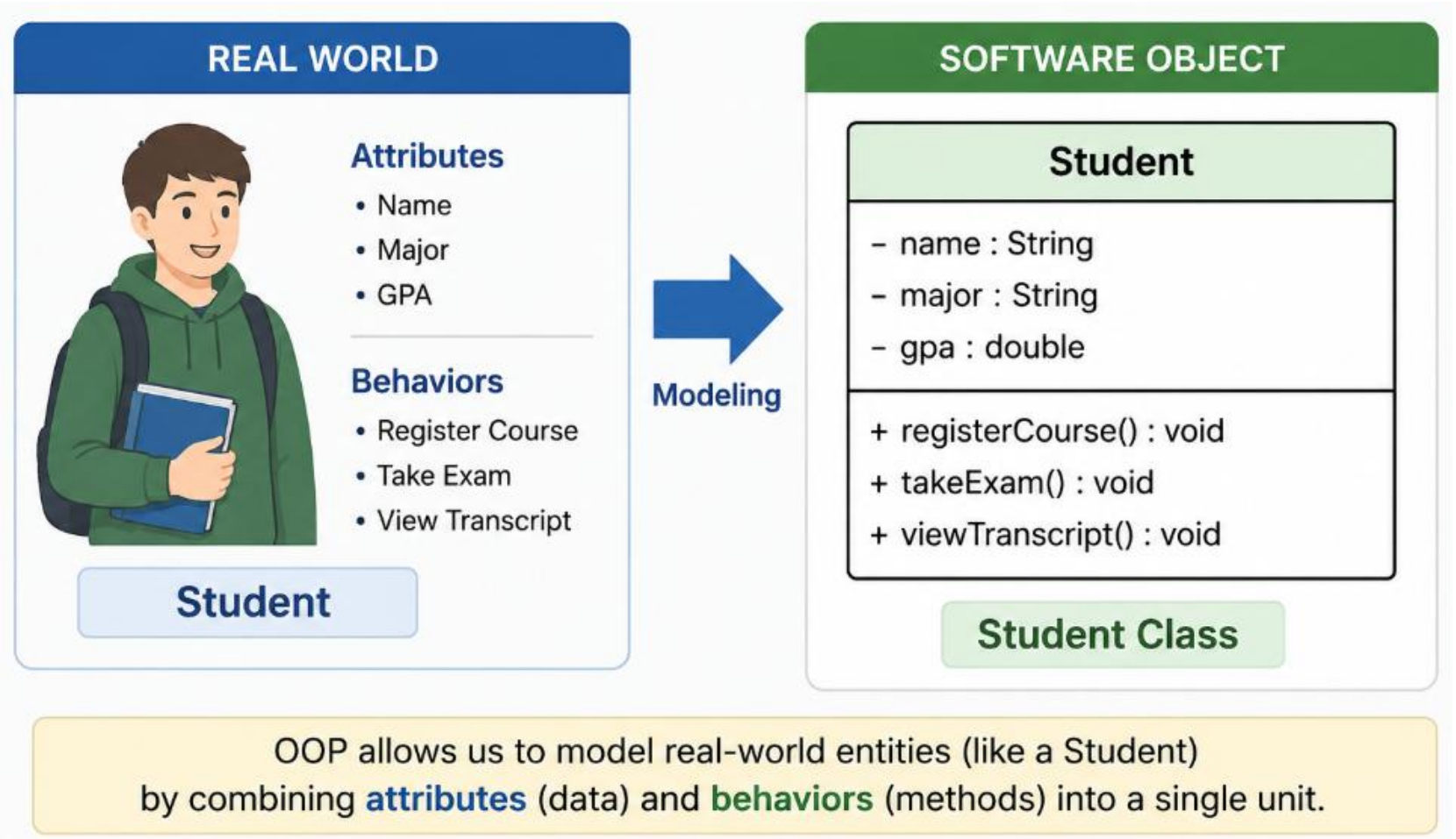
Procedural Programming

- Focuses on:
  - Functions
  - Procedures
  - Sequential execution

Challenges

- Data and functions are separate
- Difficult maintenance
- Code duplication
- Poor scalability
- Hard to collaborate in large projects

03 Real-World Modeling



03 What is Object-Oriented Programming?

OOP Definition

- Object-Oriented Programming (OOP) is a programming paradigm that organizes software around objects.
- An object contains:
  - Data (Attributes)
  - Behavior (Methods)
- OOP combines data and behavior into a single unit.

04 Benefits of OOP

- Maintainability
  - Easier to modify and manage code
- Reusability
  - Reuse existing classes
- Scalability
  - Easy to add new features
- Collaboration
  - Supports team-based development
- Real-World Modeling
  - Software structure matches real-world concepts

Class and Object

What is a Class?

- A Class is a blueprint or template used to create objects.
- It defines:
  - Attributes (Data)
  - Methods (Behavior)

```
class Student {
    String name;
    int age;

    void study() {
        System.out.println("Studying...");
    }
}
```

Class and Object

What is an Object?

- Object = Instance of a Class
- An object is a real entity created from a class.
- Creating Objects

```
Student s1 = new Student();
Student s2 = new Student();
```

Class	Object
Blueprint	Actual Instance
Definition	Real Entity
No Memory Allocated	Memory Allocated
One Class	Many Objects

Class and Object

What is an Object?

- Object = Instance of a Class
- An object is a real entity created from a class.
- Creating Objects

```
Student s1 = new Student();
Student s2 = new Student();
```

Class	Object
Blueprint	Actual Instance
Definition	Real Entity
No Memory Allocated	Memory Allocated
One Class	Many Objects

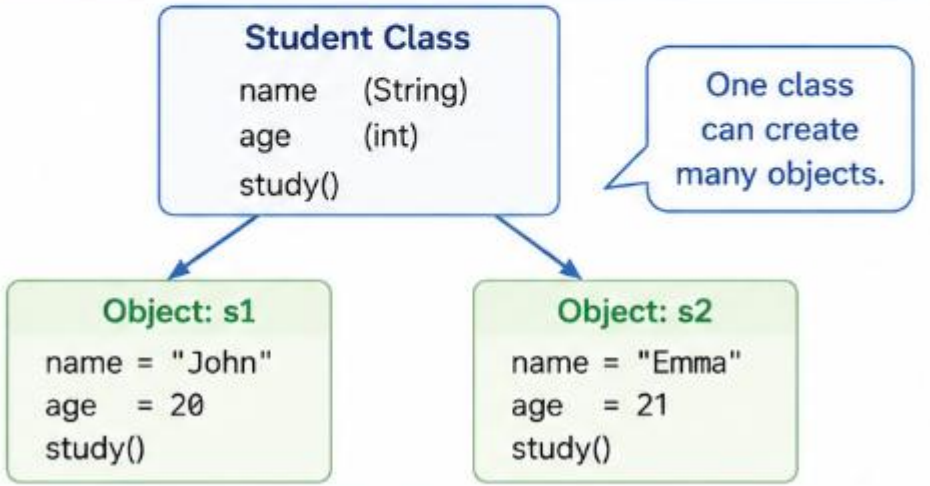
04 Class and Object

	Class	Object
Meaning	Blueprint or template	Actual instance of a class
Definition	Defines attributes and methods	Has actual values for attributes
Memory	No memory allocated	Memory is allocated
Quantity	One class	Many objects can be created
Example	Student (class)	s1, s2, s3 (objects)

Example

```
class Student {
    String name;
    int age;

    void study() {
        System.out.println("Studying...");
    }
}
// Creating objects
Student s1 = new Student();
Student s2 = new Student();
```



Class is the blueprint. Object is the real thing.

Object State and Behavior

- Every object has:
  - State (Attributes)

    name = "John"

    age = 20
  - Behavior (Methods)

    study()

    takeExam()

```
Student s1 = new Student();

s1.name = "John";
s1.age = 20;

s1.study();
```

Why Use Classes and Objects?

- Every object has:
  - State (Attributes)

    name = "John"

    age = 20
  - Behavior (Methods)

    study()

    takeExam()

```
Student s1 = new Student();

s1.name = "John";
s1.age = 20;

s1.study();
```

05 Component-Based Design (CBD)

Concept

- Component-Based Design (CBD) is a software development approach that builds applications by assembling reusable and independent software components.
- A component is a self-contained unit that provides specific functionality through well-defined interfaces while hiding its internal implementation.
- Key Characteristics
  - Reusability
  - Modularity
  - Encapsulation
  - Replaceability
  - Maintainability
  - Loose Coupling

05 Component-Based Design (CBD)

Concept

- Component-Based Design (CBD) is a software development approach that builds applications by assembling reusable and independent software components.
- A component is a self-contained unit that provides specific functionality through well-defined interfaces while hiding its internal implementation.

05 Component–Based Design (CBD)

characteristics

1. Reusability

Components are designed for reuse in multiple applications, reducing development time and cost.

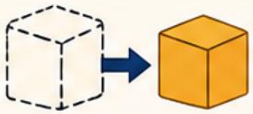
2. Modularity

The system is divided into independent, self-contained modules (components).

3. Encapuslation

Each component hides its internal details and exposes only necessary interfaces.

4. Replaceability



Components can be replaced with another compatible component without affecting the whole system.

5. Maintainability

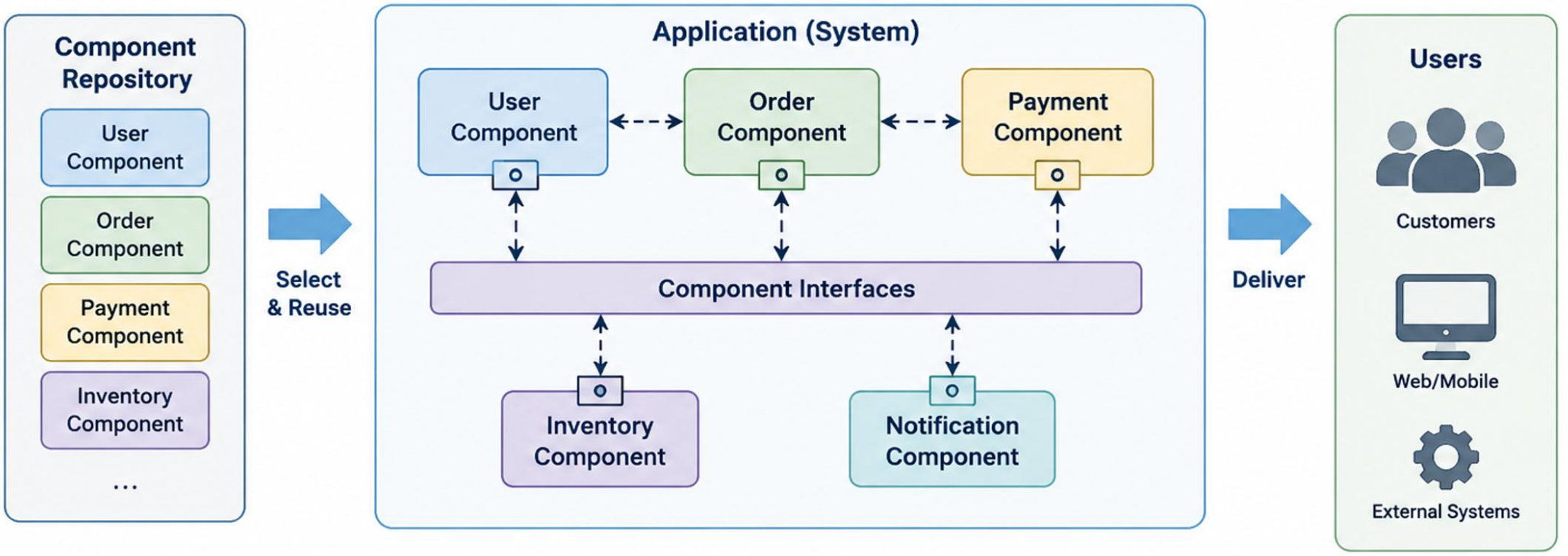
Components are easy to update, test, and maintain independently, improving productivity.

6. Loose Coupling

Components interact through well-defined interfaces with minimal dependencies.

Component-Based Design (CBD)

CBD Architecture



CBD Key Elements

Component
Reusable, self-contained unit providing specific functionality

Interface
Defines the services provided by a component

Connector
Enables communication and interaction between components

Repository
Stores and manages reusable components

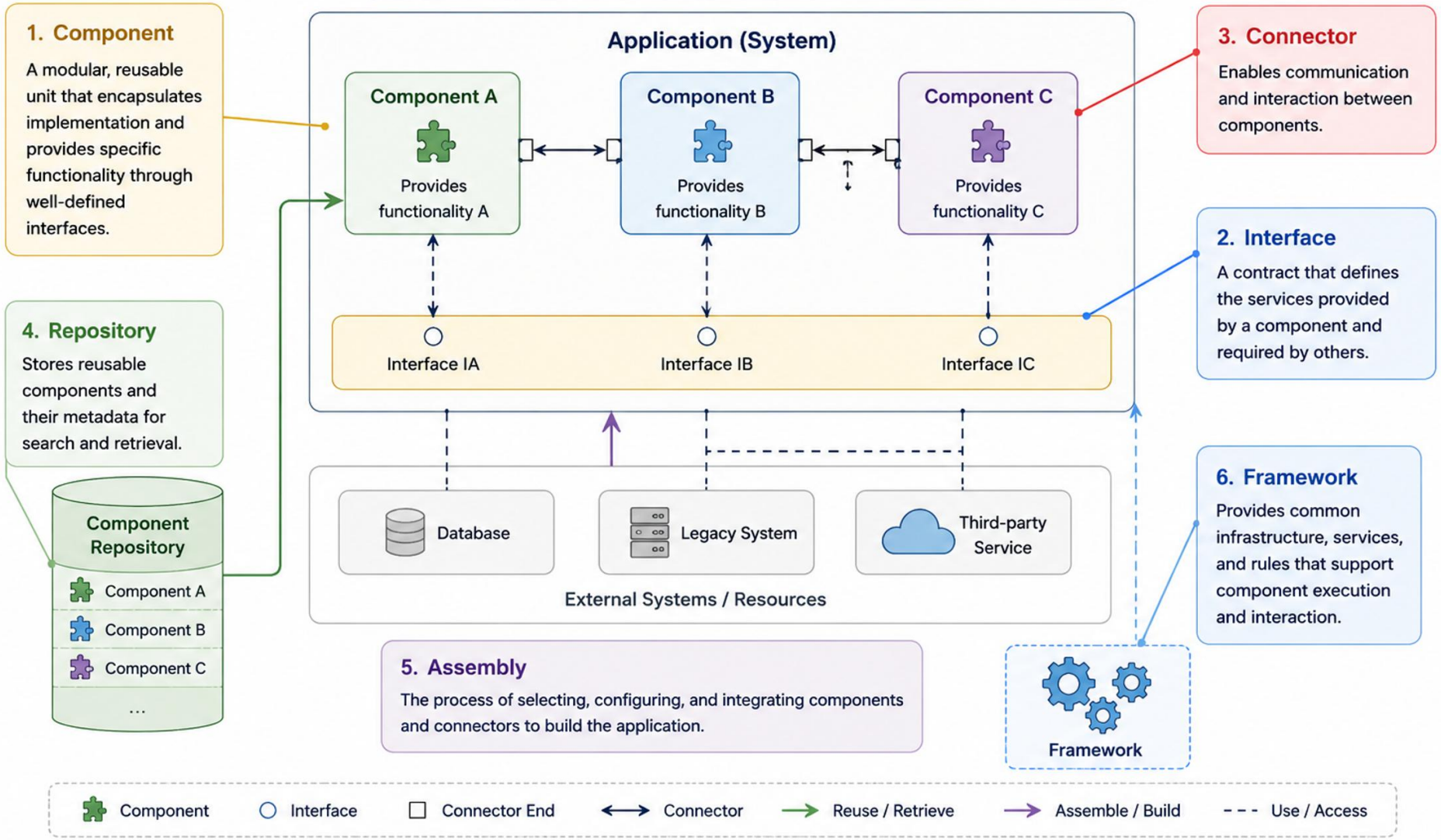
Assembly
Integrates selected components into the system

Framework
Provides infrastructure and common services for components

Component–Based Design (CBD)

Main elements

CBD builds systems by assembling reusable components.
The system is composed of the following main elements.



Component–Based Design (CBD)

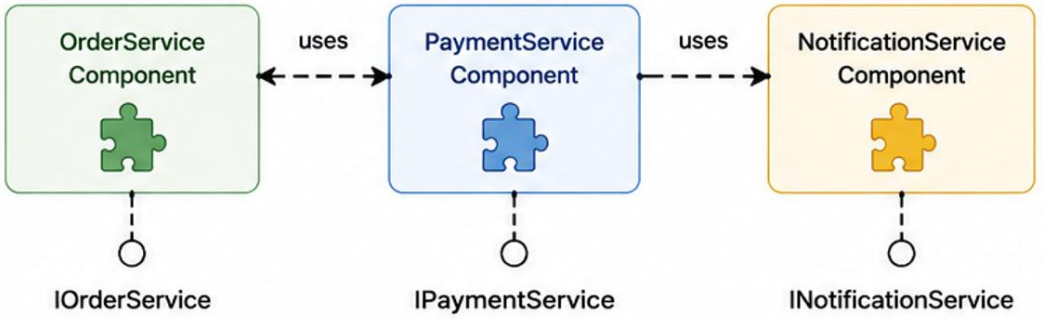
CBD development process

	1 Requirements Analysis	2 Component Identification	3 Component Selection	4 Component Adaptation	5 Component Assembly	6 Testing & Deployment
Process Steps	• Analyze business goals and requirements • Identify required functionalities and constraints	• Identify candidate components • Classify into reusable and new components • Define component interfaces	• Search component repositories • Evaluate components (functionality, quality, compatibility, cost) • Select appropriate components	• Customize or configure selected components • Resolve interface mismatches • Wrapper/adapter development if needed	• Integrate components through connectors and interfaces • Configure component interactions • Build the system	• Test component interactions and system behavior • Deploy the system • Monitor and maintain
Key Activities (example)	• Stakeholder interview • Use case analysis • Requirements specification	• Domain analysis • Identify reusable components • Define interfaces	• Repository search • Component evaluation • Proof of concept	• Parameter configuration • Interface adaptation • Data mapping	• Connect components • Resolve dependencies • System integration	• Integration testing • System testing • Deployment
Artifacts / Outputs	Requirements Document	Component Model & Interface Spec	Selected Component List	Adapted Component & Wrappers	Integrated System (Build)	Deployed System & Test Report

05 Component–Based Design (CBD)

CBD example

Component Structure



1. Interface (Contract)

```java
public interface IPaymentService {

    boolean processPayment(double amount);

}

```

2. Component Implementation

```java
public class CreditCardPayment implements IPaymentService {

    @Override
    public boolean processPayment(double amount) {
        System.out.println("Payment completed: " + amount);
        return true;
    }
}
```

3. Component Usage (Client)

```java
public class OrderService {
    private IPaymentService paymentService;

    public OrderService(IPaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder(double amount) {
        boolean result = paymentService.processPayment(amount);
        System.out.println("Order result: " + result);
    }
}
```

4. How to Use

```java
IPaymentService paymentService = new CreditCardPayment();
OrderService orderService = new OrderService(paymentService);
orderService.placeOrder(150.0);

```

Key Points

- Components provide functionality through Interfaces.
- Clients depend on Interfaces, not on concrete implementations.
- Implementation can be easily replaced without changing the client.

CBD advantage and disadvantage

Advantages	Disadvantages
Faster development	Component compatibility issues
Reusability	Integration complexity
Easier maintenance	Version management challenges
Reduced development cost	Vendor dependency
Improved quality	Performance overhead

06 Entity Relational Diagram(ERD)

Concept

- An Entity Relationship Diagram (ERD) is a visual modeling technique used to represent the structure of data within a system.
- It illustrates entities, attributes, and relationships, providing a blueprint for database design.
- ERDs help analysts, designers, and developers understand how data is organized and connected before implementing a database.

Entity Relational Diagram(ERD)

Benefits

- Understand business data requirements
- Design databases systematically
- Reduce data redundancy
- Improve data consistency and integrity
- Facilitate communication between stakeholders
- Serve as documentation for future maintenance

Entity Relational Diagram(ERD)

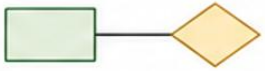
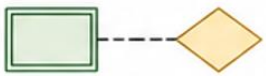
Main Components

Component	Description	Example
Entity	A real-world object or concept	Customer, Product
Attribute	Property of an entity	CustomerID, Name
Primary Key (PK)	Unique identifier	CustomerID
Foreign Key (FK)	Reference to another entity	CustomerID in Order
Relationship	Association between entities	Customer places Order
Cardinality	Number of participating instances	1:1, 1:N, M:N

06

Entity Relational Diagram(ERD)

Main component

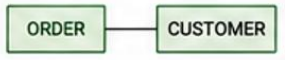
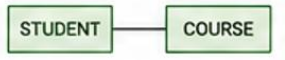
1. ERD NOTATION			
Symbol	Name	Description	Example
	Entity	A real-world object or concept.	CUSTOMER
	Attribute	A property or characteristic of an entity.	Name
	Key Attribute (Primary Key)	An attribute that uniquely identifies an entity.	CustomerID
	Derived Attribute	An attribute whose value is derived from other data.	Age (derived)
	Relationship	An association between two or more entities.	places
	Connection	A line that connects entities and relationships.	
	Multivalued Attribute	An attribute that can have multiple values.	PhoneNo (multiple)
	Weak Entity	An entity that cannot be uniquely identified without another entity.	DEPENDENT
	Identifying Relationship	A relationship that identifies a weak entity.	

06

Entity Relational Diagram(ERD)

Cardinality

2. CARDINALITY (RELATIONSHIP TYPES)

Type	Notation (Chen)	Notation (Crow's Foot)	Example	Description
One-to-One (1:1)				Each entity instance on one side is associated with at most one instance on the other side.
One-to-Many (1:N)				One entity instance on the "1" side can be associated with many instances on the "N" side.
Many-to-One (N:1)				Many entity instances on the "N" side are associated with one instance on the "1" side.
Many-to-Many (M:N)				Many instances on one side can be associated with many instances on the other side.

Notes

- Cardinality defines the number of instances that can participate in a relationship.
- Chen notation uses diamonds and numbers (1, N, M).
- Crow's Foot notation uses symbols: | (one), < (many), > (many), and combinations.

Crow's Foot Symbols

|	One and only one
<	Many
|<	One or many (optional many)
o|	Zero or one (optional one)
o<	Zero or many (optional many)

Example with Optionality

Entity Relational Diagram(ERD)

Create Process

1 Identify Entities

Identify the main entities (objects or concepts) that exist in the system.

Example

- CUSTOMER
- ORDER
- PRODUCT
- PAYMENT

2 Identify Attributes

Determine the attributes (properties) for each entity.

Example

CUSTOMER
- CustomerID (PK)
- Name
- Email
- Phone

ORDER
- OrderID (PK)
- OrderDate
- CustomerID (FK)

3 Define Primary Keys

Choose a primary key (PK) for each entity to uniquely identify records.

Example

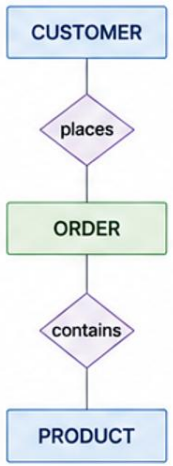
CUSTOMER
CustomerID (PK)
Name
Email
Phone

ORDER
OrderID (PK)
OrderDate
CustomerID (FK)

4 Identify Relationships

Identify the relationships between entities and give them meaningful names.

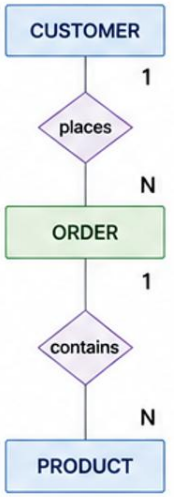
Example



5 Define Cardinalities

Define the cardinality (1:1, 1:N, N:1, M:N) for each relationship.

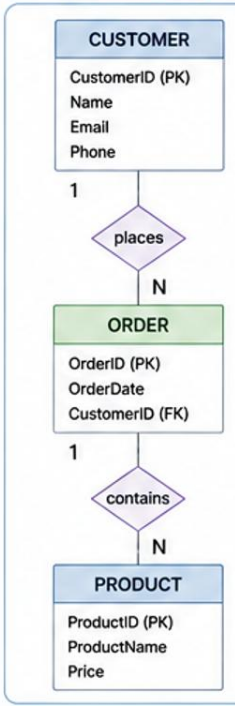
Example



6 Review & Refine

Review the ERD for accuracy, remove redundancies, and improve the model.

Example



07 Unified Modeling Language(UML)

Concepts

- UML (Unified Modeling Language) is a standardized visual modeling language used to analyze, design, and document software systems.
- t helps stakeholders understand system structure, behavior, and interactions before implementation.
- Purpose
  - Visualize system architecture
  - Analyze requirements
  - Design software solutions
  - Improve communication among stakeholders
  - Document software systems

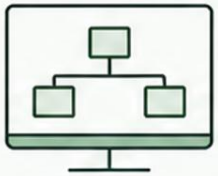
Unified Modeling Language(UML)

Key Characteristics

1 Standardized

Uses internationally recognized notation and semantics.

2 Visual



Represents system structure and behavior graphically for easy understanding.

3 Object-Oriented

Designed based on object-oriented concepts such as classes, objects, inheritance, and polymorphism.

4 Platform Independent

Can be applied regardless of programming language or development platform.

5 Comprehensive

Provides a wide range of diagrams to model structure, behavior, and interactions.

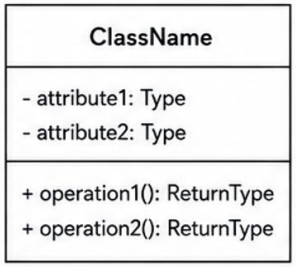
6 Extensible

Can be extended with new mechanisms and custom profiles for specific domains.

Unified Modeling Language(UML)

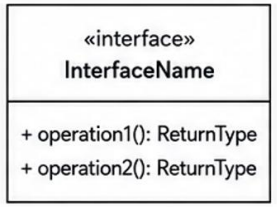
Main components

1. Class



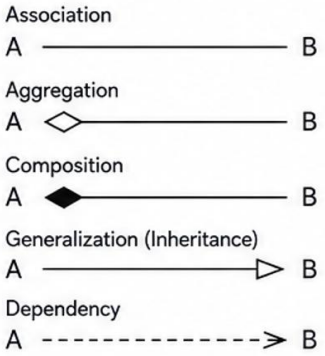
A basic building block that represents the structure of a system, including attributes and operations.

2. Interface



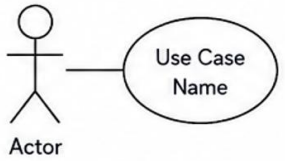
A contract that defines a set of operations (behaviors) that a class must implement.

3. Relationship



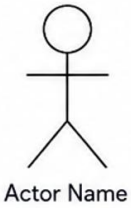
Relationships show how elements are connected.

4. Use Case



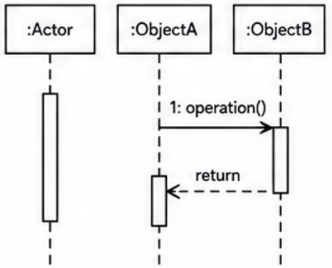
Represents a functionality (or service) provided by the system from the user's perspective.

5. Actor



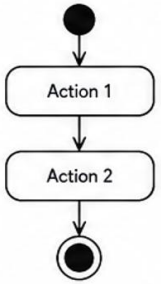
An external entity that interacts with the system, such as a person, another system, or external entity.

6. Sequence Diagram



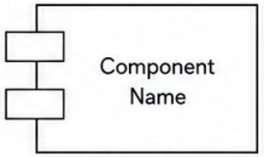
Shows the time order of messages exchanged between objects.

7. Activity Diagram



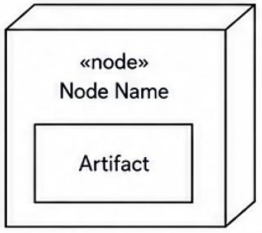
Models the workflow of activities and control flow within a process.

8. Component Diagram



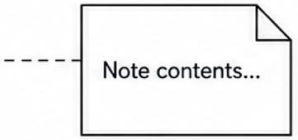
Represents the physical and software components of a system.

9. Deployment Diagram



Shows the physical deployment architecture, nodes, and artifacts.

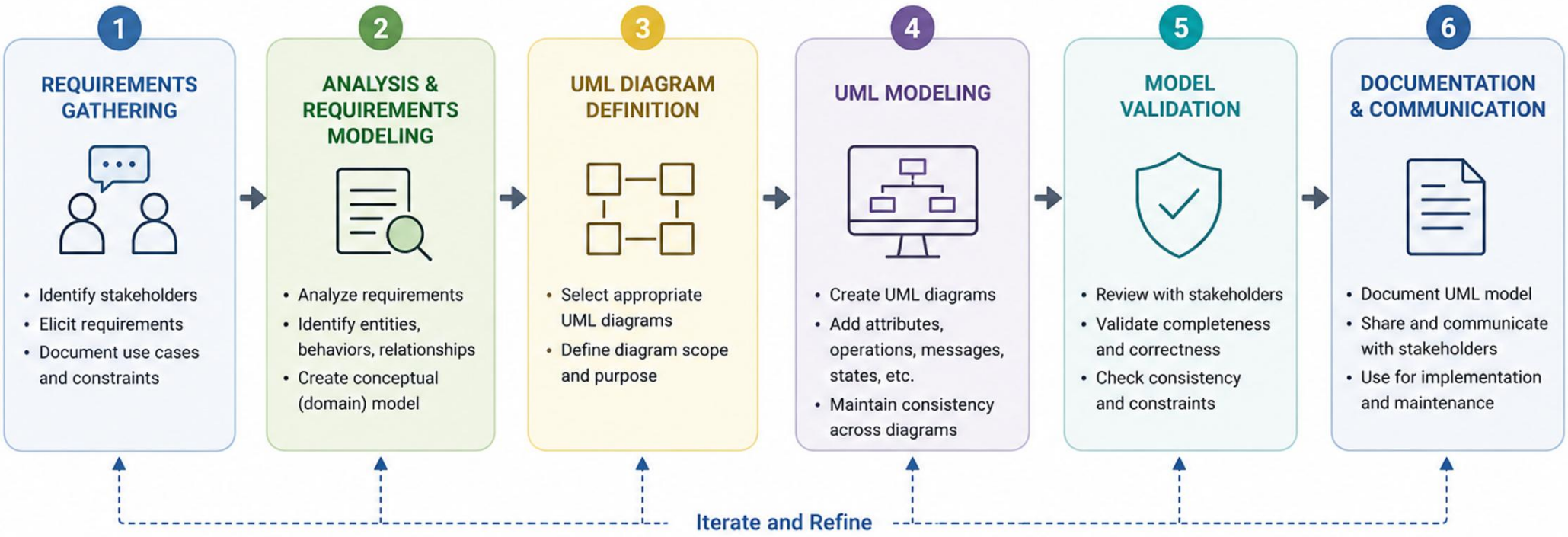
10. Note



Provides comments or constraints to describe elements or diagrams.

Unified Modeling Language(UML)

Create process



08 Purpose of Training

TODAY'S IT LANDSCAPE

CLOUD
Elastic, Scalable Infrastructure

AI
Intelligence & Automation

BIG DATA
Data-Driven Insights

1. UNDERSTAND KUBERNETES PLATFORM
Understand Kubernetes as the most important IT platforma for Cloud, AI, and Big Data era.

kubernetes
CLOUD-NATIVE PLATFORM

- CONTAINER MANAGEMENT
- SERVICE DISCOVERY
- SCALING & SELF-HEALING
- SECURITY
- MONITORING & OBSERVABILITY

2. UNDERSTAND AND APPLY MSA ARCHITECTURE
Design, develop, and operate Microservices-based applications.

- USER SERVICE
- ORDER SERVICE
- PAYMENT SERVICE
- INVENTORY SERVICE
- NOTIFICATION SERVICE
- ...

3. APPLY IN REAL-WORLD SCENARIOS
Use AI, Big Data, and Cloud-Native applications on Kubernetes-based MSA.

- **AI APPLICATIONS**: LLM, ML Services, Intelligent Apps
- **BIG DATA PIPELINES**: Data Ingestion, Processing, Analytics
- **CLOUD-NATIVE APPS**: Web/Mobile Apps, APIs, Event-Driven

4. HANDS-ON PRACTICE & KNOWLEDGE ACCUMULATION
Labs, Projects, Best Practices, and Experience Sharing

FINAL GOAL

CAMBODIA MSA CAPABILITY BUILDING

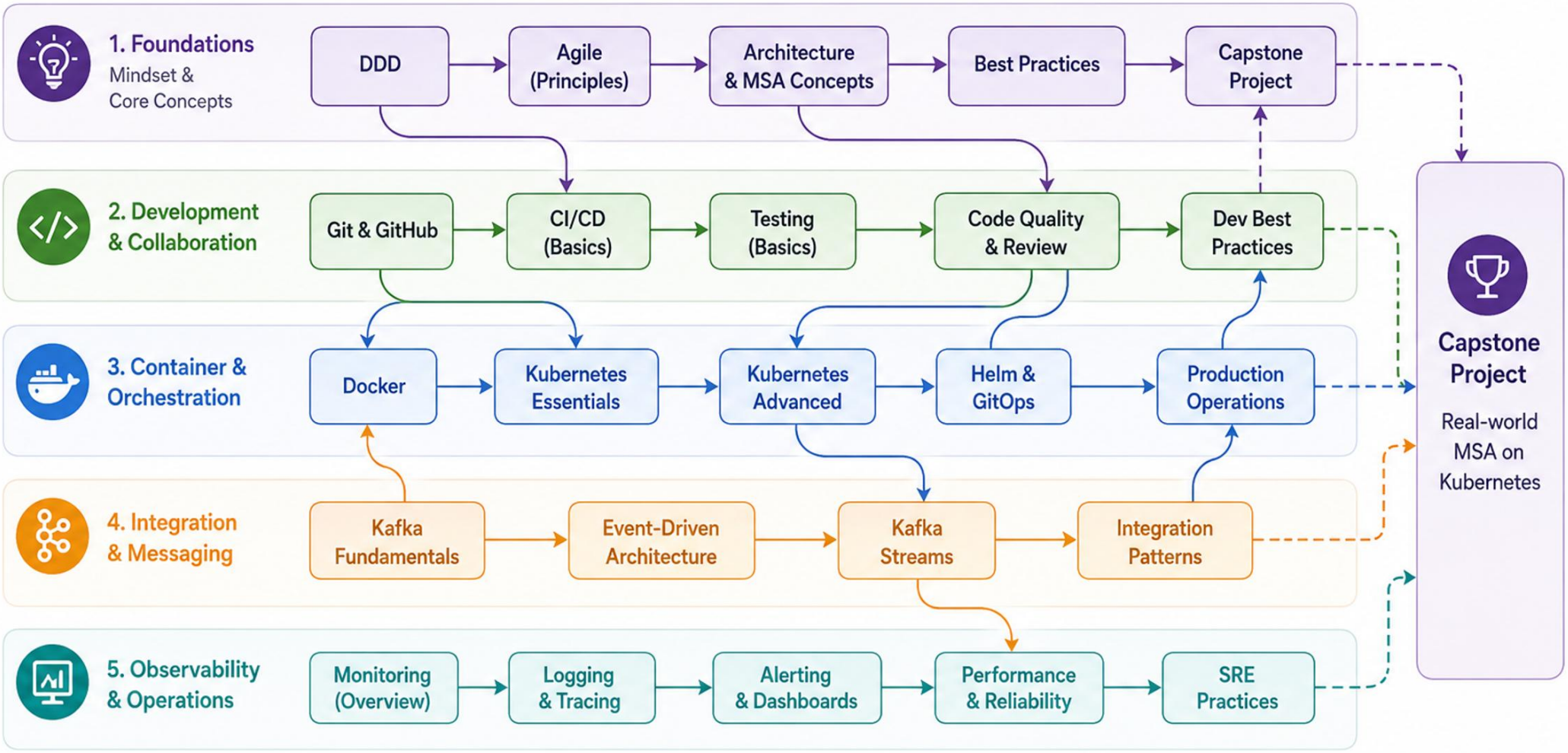
INTERNALIZE MSA TECHNOLOGY
Build local expertise and technical capabilities

ACHIEVE SELF-RELIANCE
Operate, maintain, and innovate independently

SUSTAINABLE GROWTH
Empower digital transformation and economic development

BUILD MSA TECHNOLOGY CAPABILITY AND SELF-RELIANCE IN CAMBODIA
Empower people, strengthen systems, and drive sustainable digital transformation.

09 Training Process



Thank You